© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:42

2 Grundlagen von Datenbanksystemen

2.1 Herausforderungen und Zielsetzung

2.1.1 Problemfelder im Datenmanagement

Im betriebswirtschaftlichen Umfeld ist der Einsatz von Informationssystemen für die effiziente Unterstützung von Geschäftsprozessen unabdingbar. Die Aktionsfähigkeit und der Erfolg einer Organisation hängen oft direkt von der Verfügbarkeit und der Qualität der hierfür eingesetzten Daten ab. Dem Einsatz von Datenbanksystemen als technologischer Lösung kommt in diesem Zusammenhang eine ganz besondere Bedeutung zu. Das Management und die Organisation relevanter Daten müssen wohl überlegt, planvoll und systematisch erfolgen. Datenorganisation und Datenmanagement haben daher in den Unternehmen immer mehr an Bedeutung gewonnen und wurden als zentrale Rollen etabliert.

Die manuelle Verwaltung von Daten in individuell strukturierten und gespeicherten Dateien, meist auf unterschiedlichen Rechnersystemen, genügt den heutigen Anforderungen eines nachhaltigen Datenmanagements nicht. Ohne ein effizientes Datenmanagement, die systematische Datenorganisation und eine Datenhaltung entstehen essentielle Probleme in der IT-Landschaft eines Unternehmens.

In der Entwicklung von Anwendungssystemen können hinsichtlich der Organisation der relevanten Daten ein funktionsorientierter oder ein datenorientierter Ansatz verfolgt werden. Die Ansätze unterscheiden sich wie folgt:

- **Funktionsorientierter Ansatz**: Im Vordergrund stehen die Funktionen für die Bewerkstelligung der zu realisierenden Aufgaben im Problembereich; die zu nutzenden Daten sind nachgelagert. Beim diesem Ansatz konzentriert man sich bei der Entwicklung von Anwendungen zunächst auf die für eine Aufgabe relevanten Funktionen; erst in einem zweiten Schritt werden die erforderlichen Daten ermittelt und definiert.
- **Datenorientierter Ansatz**: Im Vordergrund steht die Erarbeitung einer logischen Gesamtsicht auf die relevanten Unternehmensdaten. Kennzeichen ist ein schrittweiser, systematischer Übergang von einer an Einzelanwendungen ausgerichteten Datenorganisation hin zu einer anwendungsübergreifenden, an den Aufgaben des Gesamtunternehmens ausgerichteten Organisation und Verwaltung von Daten.

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:42

Die Entwicklung von Anwendungssystemen nach dem funktionsorientierten Ansatz erzeugt sogenannte Insellösungen, die signifikante Nachteile mit sich bringen. Abbildung 2.1 verdeutlicht an einem Beispiel eine solche Situation. Ein Unternehmen hat für drei Funktionsbereiche wie beispielsweise Einkauf, Fakturierung und Vertrieb, jeweils Anwendungsprogramme I, II, III, hier kurz als AP-1, AP-2 und AP-3 bezeichnet, erstellt. Für jedes dieser Anwendungsprogramme ist eine Datei, hier Datei A, B und C, eingerichtet worden, die für die permanente Speicherung der Anwendungsdaten, wie beispielsweise Kundendaten und Artikeldaten, dienen sollen.

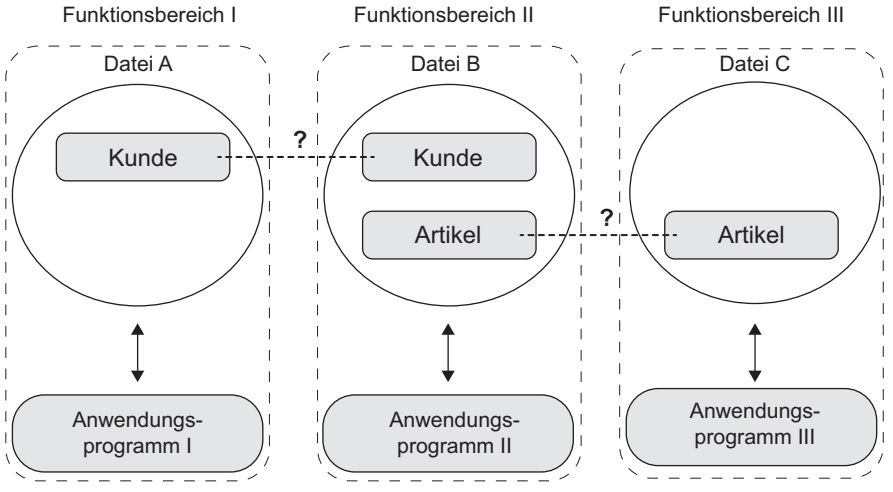


Abb. 2.1: Klassische Dateiorganisation

Die Datenhaltung nach einer Dateiorganisation hat Nachteile, die einen Einsatz von Datenbanksystemen propagieren:

- **Hohe Redundanz** von Daten:
  Die Speicherung der Anwendungsdaten in den nach Funktionen orientierten Dateien bedingt eine Mehrfachspeicherung von gleichen Daten. Hieraus resultiert bei Änderung von Datenbeständen in einer Datei die Gefahr der Inkonsistenz von Daten.
  Beispiel: Jedes der Anwendungsprogramme AP-1 und AP-2 hat die jeweils zur Funktionserfüllung notwendigen Kundendaten in einer spezifischen Datei A bzw. Datei B gespeichert. Sobald sich die Adresse eines Kunden ändert, müssen in allen Dateien im Unternehmen die entsprechenden Kundenadressen durchsucht und verändert werden. Oftmals

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:42

ist nicht genau bekannt, in welchen Dateien überall Kundendaten verwaltet werden.

- **Enge Kopplung** von Programm und Daten:
  Die Programmlogik ist abhängig von der Struktur der Daten bzw. von der dateiinternen Speicherorganisation. Damit führt die Änderung der Datenstruktur, beispielsweise bei Erweiterung der Daten um neue Merkmale, unmittelbar zu einer notwendigen Anpassung der Programmlogik.
  Beispiel: Die Kundendaten müssen zwingend um das Merkmal *Mailadresse* erweitert werden. Die Struktur von Datei A und Datei B muss jeweils verändert werden, um zukünftig das Attribut *Mailadresse* speichern zu können. Wegen dieser Strukturänderung in der Datei muss auch die Programmlogik in AP-1 sowie in AP-2 hinsichtlich der Zugriffsfunktion auf die Daten angepasst werden. Die Anpassung der Programmlogik erfordert Implementierungsaufwand und bedingt einen neuen Funktionstest auf Korrektheit.
- **Mehrfachimplementierung** von Standard-Zugriffsroutinen:
  In den verschiedenen Anwendungssystemen müssen bestimmte Datenverarbeitungsroutinen, wie beispielsweise das Einfügen von Datensätzen oder das Suchen nach bestimmten Informationen, mehrfach programmiert werden. Dies ist wirtschaftlich und auch im Hinblick auf die Wartung sehr ungünstig.
  Beispiel: In der Datei A sind die Kundendaten zeilenweise organisiert, in der Datei B spaltenweise. Die Funktion zum Einfügen eines neuen Kunden muss im Anwendungsprogramm AP-1 programmiert werden und kann aber aufgrund der unterschiedlichen Speicherorganisation nicht auf AP-2 übertragen werden. In AP-2 muss eine eigene Funktion für das Einfügen von Kundendatensätzen implementiert werden.

2.1.2 Historische Entwicklung aus Sicht der Daten

Die Notwendigkeit zum Einsatz und der Nutzen von Datenbanksystemen, aber auch die veränderten Herausforderungen für das Datenmanagement in den Unternehmen, können anhand der Entwicklungsstufen in der Informationsverarbeitung aus Sicht der Daten verfolgt werden. Die Entwicklungsstufen sind geprägt vom systematischen Übergang von einer an Einzelanwendungen ausgerichteten Datenorganisation hin zu einer anwendungsübergreifenden, an den Aufgaben des Gesamtunternehmens ausgerichteten Organisation und Verwaltung von Daten.

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:42

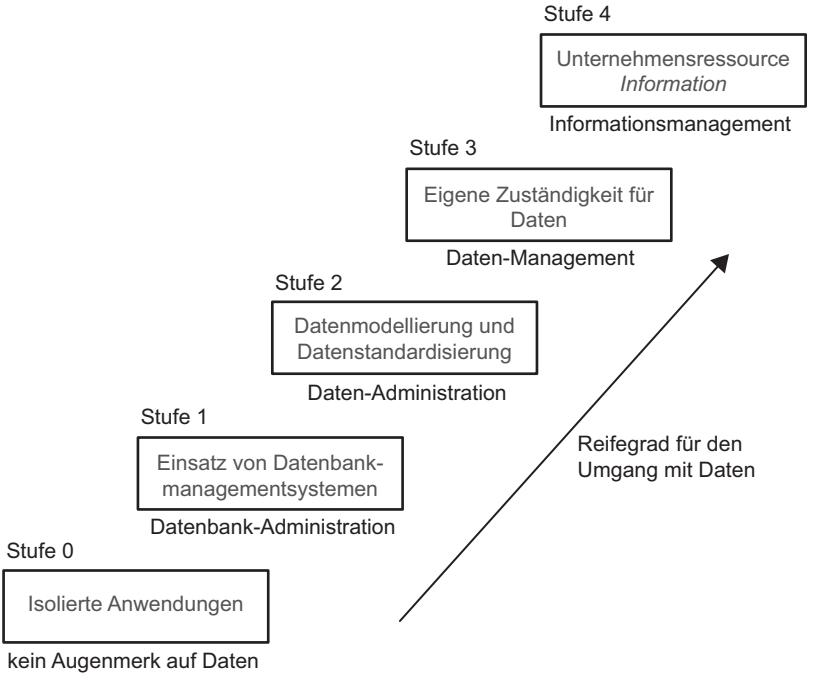


Abb. 2.2: Entwicklung der Informationsverarbeitung aus Datensicht

- **Stufe 0**: Isolierte Anwendungen
  Die Ausgangssituation der Informationsverarbeitung ist geprägt durch isolierte Anwendungsprogramme, die nach dem funktionsorientierten Ansatz in Insellösungen aufgeteilt waren (▶ Abb. 2.1). Die Daten standen nicht im Vordergrund der Betrachtung.
- **Stufe 1**: Einsatz von Datenbanksystemen
  Mit dem Einsatz von Datenbanksystemen sollten die wesentlichen Nachteile der engen Kopplung von Programm und Daten sowie der redundanten Datenhaltung aufgelöst werden. Mit der Einführung des Konzeptes der Datenbanksysteme wurde in den Unternehmen erstmals auch die Rolle des **Datenbankadministrators** geschaffen. Der Datenbankadministrator hat sich in seiner Aufgabe sehr stark auf die technologische Aufgabe und Lösung im Einsatz eines Datenbanksystems fokussiert.
- **Stufe 2**: Datenmodellierung und Datenstandardisierung
  Selbst der technologische Einsatz von Datenbanksystemen hat die Grundidee einer an der logischen Gesamtsicht des Unternehmens orientierten Datenverwaltung nicht hinreichend realisieren lassen. Die Erkenntnis, dass eine konzeptionelle Erarbeitung eines Datenmodelles

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:42

als logische Gesamtsicht der Unternehmensdaten die Voraussetzung für ein Datenbanksystem sein muss, ließ das Augenmerk zunehmend auf die Datenmodellierung und Datenstandardisierung richten. Damit verbunden ist die Rolle eines **Datenadministrators** etabliert worden. Der Datenadministrator zeichnet sich für die fachliche, konzeptionelle Datenmodellierung im Unternehmen verantwortlich.

- **Stufe 3**: Datenmanagement im Unternehmen
  Eine logische Gesamtsicht der Unternehmensdaten verlangt eine abteilungs- und funktionsübergreifende Perspektive und Verantwortung bei der Erstellung von Datenmodellen. Entsprechend bedarf es der Einführung der Rolle eines **Datenmanagements** mit einer eigenen Zuständigkeit für die Daten im Unternehmen.
- **Stufe 4**: Informationsmanagement
  Spätestens mit der Erkenntnis, dass Daten und Informationen eine essentielle Unternehmensressource darstellen, wurde das **Informationsmanagement** im Unternehmen eingeführt und im Management als Aufgabe verankert.

2.1.3 Zielsetzung für den Einsatz von Datenbanksystemen

Fokussierung auf den datenorientierten Ansatz

Um zentrale Schwachstellen bei der konventionellen dateiorientierten Datenorganisation zu lösen, wurde das Konzept der Datenbanksysteme entwickelt und als technologische Lösung eingeführt. Mit einem Datenbanksystem werden alle notwendigen Anwendungsdaten durch eine eigene Software zentral verwaltet und organisiert. Den Anwendungen und Benutzern werden die jeweils gewünschten Daten über wohldefinierte Schnittstellen bereitgestellt. Datenbanksysteme verhelfen zu mehr Leistungsfähigkeit sowohl im Umgang mit großen Datenmengen als auch in der Abdeckung des Datenbedarfs in den Geschäftsprozessen.

Mit dem Konzept von Datenbanksystemen wird das Ziel verfolgt, eine Abkehr vom funktionsorientierten Ansatz der Anwendungsentwicklung hin zu einem datenorientierten Ansatz zu bewältigen. Die Trennung der logischen Datensicht von der physischen Speicherung ist Grundlage der angestrebten Entkopplung von Programm und zugehörigen Daten. Dies bewirkt einen systematischen Übergang von einer an Einzelanwendungen ausgerichteten Datenorganisation hin zu einer anwendungsübergreifenden und an den Zielen des Gesamtunternehmens orientierten Verwaltung und Organisation von Daten. Die Datenorganisation und das Datenmanagement erlangt damit in den Unternehmen eine zentrale Bedeutung.

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:42

Datenbanksysteme sind heute ein elementarer Bestandteil der IT-Landschaft in den Unternehmen zur effizienten Verwaltung großer Datenmengen und einer wohlorganisierten Datenhaltung. Sie erleichtern und unterstützen den Umgang mit Daten, schaffen Transparenz und Übersichtlichkeit in großen Datenbeständen. Zukünftig immer wichtiger wird die Erfüllung von Anforderungen an die IT-Governance und IT-Compliance, die durch den Einsatz von Datenbanksystemen zu einem großen Teil realisierbar sein werden.

Vor- und Nachteile des Datenbankeinsatzes

Die Verwendung von Datenbanksystemen zur Datenhaltung lohnt sich in der Regel erst für größere Datenbestände, die in mehreren Bereichen bzw. an verschiedenen Arbeitsplätzen benötigt werden. Ist dies der Fall, erbringt der Einsatz eines Datenbanksystems folgende **Vorteile**:

- **Optimale Redundanz**: Die Verringerung und Elimination von Datenredundanz ist erklärtes Ziel. Falls Datenredundanz systemseitig Vorteile erbringt, muss eine systematische Kontrolle der Redundanz von Daten das Ziel sein.
- **Zentralisierte Datenhaltung**: Eine zentrale Datenbasis impliziert einen geringeren Arbeitsaufwand für die Verwaltung der Daten und schafft mehr Übersichtlichkeit. Die Einsparung von Ressourcen an den einzelnen Arbeitsplätzen wird ermöglicht.
- **Wirtschaftlichkeit**: In der Einführung und Pflege von Anwendungssystemen entstehen Nutzeneffekte aufgrund der Kopplung von Anwendungssystemen mit einer zentralisierten Datenorganisation.
- **Nutzung und Einführung von Standards**: Die Datenstandardisierung wird mit dem Einsatz von Datenbanksystemen erleichtert. Die zentrale Datenbasis schafft standardisierte und einheitliche Formate zur Datenspeicherung. Darüber hinaus können genormte Zugriffsschnittstellen auf die Daten angeboten werden. Durch standardisierte Datenformate und Zugriffsschnittstellen können Daten von verschiedenen Systemen, Anwendungen und Benutzern verwendet werden.
- **Logische und physische Datenunabhängigkeit**: Die Entkopplung von Anwendungsprogrammen und den Daten schafft eine Datenunabhängigkeit, die Änderungen und Erweiterungen von Datenstrukturen im Datenbanksystem ermöglichen, ohne dass Änderungen in den Anwendungsprogrammen notwendig werden.
- **Gewährleistung der Konsistenz und Korrektheit von Daten** sowie die Überwachung der Datenintegrität: Das Datenbanksystem sorgt für einen kontrollierten und synchronisierten Zugriff auf die zentral gespeicherten Daten. Jede Datenbankoperation transformiert damit einen

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:42

konsistenten Datenbestand in einen wiederum konsistenten Bestand an Daten. Die Vermeidung von Fehleingaben und unzulässigen Operationen wird durch Plausibilitätsprüfungen unterstützt.
- **Möglichkeit der Zugriffskontrolle** durch verschiedene Berechtigungsrollen: Damit wird der Einsatz in einem Umfeld verschiedener Benutzer mit unterschiedlichen Zugriffsberechtigungen auf die Daten des Unternehmens ermöglicht.

Der Einsatz eines Datenbanksystems bringt allerdings auch **Nachteile** in der Anwendung mit sich.

- **Wirtschaftlichkeit**: Die Kosten für erforderliche Hardware und Software-Lizenzen stehen insbesondere bei kleineren Anwendungsumgebungen in einem ungünstigen Verhältnis zum Nutzen.
- **Abhängigkeit von Fachpersonal für Administration, Pflege und Wartung**: Der Einsatz von Datenbanksystemen bedingt administrative Aufgaben (siehe Datenbankadministration), die entsprechende Fachkompetenz erforderlich machen.
- **Effizienzmangel bei spezialisierten Anwendungen**: Verwaltungssoftware der Datenbanksysteme kann harte Echtzeitbedingungen in zeitkritischen Anwendungen oftmals nicht erfüllen und weist Performanceprobleme bei zeit- und rechenintensiven Anwendungen auf.
- **Verwundbarkeit durch Zentralisierung aller Daten**: Ein technischer Ausfall des Datenbanksystems führt dazu, dass keine Anwendungssysteme mehr bedient und mit Daten versorgt werden können.

Integrationseffekt der Daten

Datenbanksysteme werden oft als Integrationswerkzeuge beschrieben, weil von der anwendungsübergreifenden, zentralen Datenbasis im Unternehmen eine integrierende Wirkung ausgeht (▶ Abb. 2.3).

Die Anwendungssysteme im Unternehmen werden alle von der gleichen, zentralen Datenbasis versorgt. Beispielsweise nutzen die Anwendungen *Produktion* und *Verkauf* dieselben *Artikelstammdaten*. Veränderungen an den Dateninhalten eines Objektes werden automatisch für alle nutzenden Anwendungssysteme der zentralen Datenbasis übertragen, man spricht vom **Integrationseffekt der Daten**. Beispielsweise wird durch die *Produktion* der Lagerbestand eines *Artikels* erhöht, so ist bei Abruf des verfügbaren Lagerbestands des gleichen *Artikels* der Vertriebsbeauftragte umgehend über den aktuellen Lagerbestand informiert.

Unter Ausnutzung des Integrationseffektes der Daten (▶ Abb. 2.4) sollen mit der Datenintegration insbesondere integrierte Systemlösungen gefördert werden. Eine horizontale Integration der Anwendungssysteme

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:42

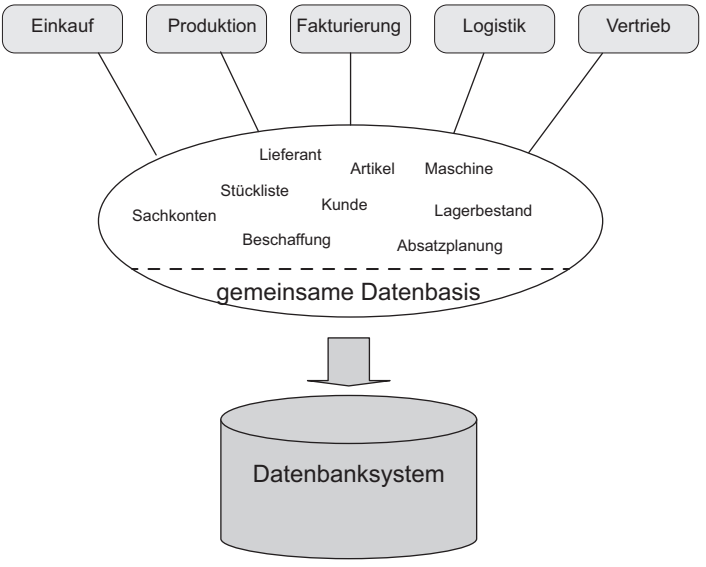


Abb. 2.3: Datenbanksysteme als Integrationswerkzeuge

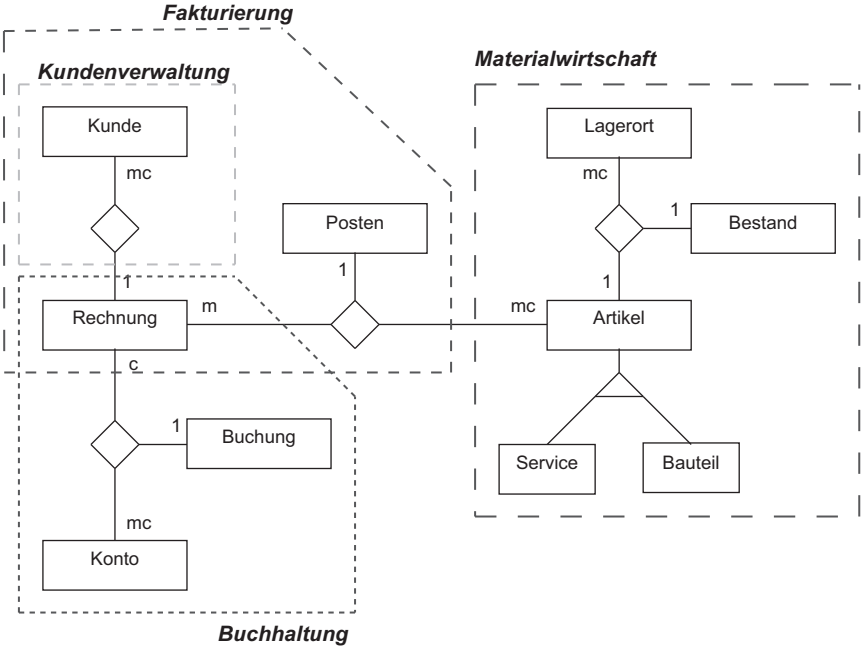


Abb. 2.4: Integrationseffekt der Daten

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:42

über die verschiedenen Funktionsbereiche eines Unternehmens hinweg, auf der Grundlage der Nutzung einer zentralen Datenbasis, wird somit ermöglicht.

2.2 Komponenten eines Datenbanksystems

2.2.1 Grundaufbau

Ein **Datenbanksystem** ist eine auf Dauer ausgelegte, effiziente Datenorganisation zur geregelten, sicheren und flexiblen Speicherung und Wiedergewinnung von Datenbeständen, die mehreren Anwendungen und Benutzern zur Verfügung gestellt werden.
Damit ein Datenbanksystem die spezifizierten Anforderungen und Aufgaben erfüllen kann, werden verschiedene Komponenten benötigt. Allgemein besteht ein Datenbanksystem aus einer oder mehreren **Datenbanken** sowie einer Software zur effizienten Verwaltung und Bereitstellung der Daten aus den Datenbanken, dem **Datenbankmanagementsystem**.

Ein **Datenbanksystem** (DBS) besteht aus folgenden zwei zentralen Komponenten:

- einer strukturierten Sammlung von Primär- und Sekundärdaten, der sogenannten **Datenbasis** oder **Datenbank** (DB), und
- einer speziellen Datenbanksoftware zur Verwaltung der Daten, dem sogenannten **Datenbankmanagementsystem** (DBMS).

Während die Datenbank-Komponente den **statischen Teil** verkörpert, ist durch das Datenbankmanagementsystem der **dynamische Teil** eines Datenbanksystems gegeben. Oft wird der Begriff Datenbank auch als Bezeichnung für ein komplettes Datenbanksystem verwendet; dies ist jedoch im eigentlichen Sinne nicht richtig.

Ein Datenbanksystem unterstützt und erleichtert den Umgang mit Daten, indem es eine Umgebung bietet, in der Daten besser organisiert, verwaltet und kontrolliert werden können. Ein Datenbanksystem kann einen großen Umfang an Daten wie auch beliebige Typen von Datenobjekten, beispielsweise Texte, Grafiken und Videos, speichern und verwalten.

Datenbanksysteme stellen Funktionen zur Verfügung, die für das Einfügen, die Wiedergewinnung sowie für die Pflege der Daten relevant

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:42

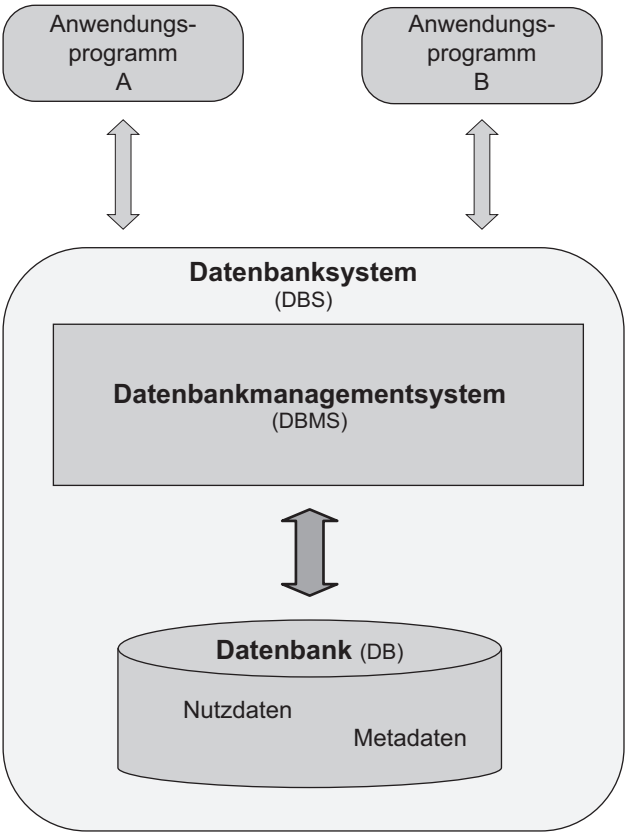


Abb. 2.5: Aufbau eines Datenbanksystems

sind. Zur Durchführung von solchen Operationen mit der Datenbank oder dem Datenbestand stellt das Datenbankmanagementsystem eine meist standardisierte Datenbanksprache, beispielsweise SQL (▶ Kap. 6), zur Verfügung.

Die letztendliche Verarbeitung der Daten erfolgt durch die Benutzer und Anwendungsprogramme, die über die vom Datenbankmanagementsystem bereitgestellte Kommunikationsschnittstelle auf die Daten der Datenbank zugreifen und diese bearbeiten können.

2.2.2 Datenbank – statische Komponente

Eine **Datenbank** (engl.: ***Data**base,* kurz *DB*) bezeichnet eine logisch zusammenhängende Sammlung von Daten für eine bestimmten Zweck.

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:42

Eine zufällige Ansammlung von Daten ist keine Datenbank, da es sich hierbei um keine logisch zusammenhängende Sammlung von Daten handelt. Daten einer Datenbank sind für einen definierten Zweck zusammengestellt und werden bestimmten Benutzern und Anwendungen zur Verfügung gestellt.

Die gesammelten Daten sind in der Datenbank persistent, also dauerhaft über die Laufzeit des Anwendungsprogrammes hinaus, gespeichert und liegen im Allgemeinen als strukturierte Datensätze vor. Dabei enthält jeder Datensatz Informationen über ein Objekt der realen Welt, beispielsweise über eine bestimmte Person oder einen Artikel. Die Datenbank enthält in der Regel nicht nur die reinen **Nutzdaten** (Primärdaten), sondern auch zusätzliche Verwaltungs- und Strukturinformationen über die Daten, die sogenannten Metadaten (Sekundärdaten).

Beispiel:
Die Datenbank enthält eine Tabelle mit den Kundendaten des Unternehmens. Die *Kundendaten*, also alle Datensätze der Tabelle, stellen die **Nutzdaten** dar. Die Information, welche Spalten mit welchen Datentypen diese Kundentabelle enthält, ist Verwaltungsinformation und wird in einem separaten Bereich der Datenbank gespeichert. Diese Informationen gehören zu den **Metadaten**.

Das Datenbanksystem stellt typischerweise einen **Datenkatalog**, auch Datenwörterbuch genannt (engl.: ***Data Dictionary***), zur Verfügung, der Verwaltungsinformationen und Metadaten über die Datenbank-Objekte, Datenbank-Benutzer und Datenbank-Ereignisse enthält. In diesem Datenkatalog sind auch Angaben über die inhaltliche Bedeutung wie auch über die Benutzung, Nutzungsrechte und Integritätsbedingungen sowie über die physische Repräsentation der in der Datenbank gespeicherten Daten enthalten.

Die strukturelle Beschreibung der Datenbank, also wie die Nutzdaten in der Datenbank abgelegt werden, wird in einer sogenannten Datenbeschreibungssprache festgelegt und durch das interne Datenmodell repräsentiert. Für die Bereitstellung einer Datenbeschreibungssprache, beispielsweise in Form von SQL, ist das Datenbankmanagementsystem zuständig.

Die interne, physische Organisation einer Datenbank wird durch das konkrete Datenbanksystem bestimmt. Die Datenbank kann beispielsweise aus einer Ansammlung von Dateien bestehen, die in einem Verzeichnis des Dateisystems auf einem Rechner verwaltet werden. Nur das Daten-

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:42

banksystem selbst kennt die Datenstruktur, welche zu den jeweiligen Daten innerhalb der Dateien führen.

Eine Datenbank kann aus mehreren Teil-Datenbanken bestehen, die sich auf unterschiedlichen Rechnern, also an unterschiedlichen Plätzen, befinden können. Logisch handelt es sich für die jeweiligen Anwendungen dennoch nur um **eine** Datenbank; hier spricht man von **verteilten Datenbanken**. Eine verteilte Datenbank und deren Inhalt werden durch ein gemeinsames Datenbankmanagementsystem verwaltet und kontrolliert.

2.2.3 Datenbankmanagementsystem – dynamische Komponente

Zur Nutzung und Verwaltung einer Datenbank und der in einer Datenbank gespeicherten Daten, wird das sogenannte **Datenbankmanagementsystem** (engl.: ***D**atab**a**se **M**anagement **S**ystem*, kurz: DBMS) benötigt. Es ist eine Software und stellt Funktionen zur Definition, Erzeugung und Manipulation von Datenbanken sowie Funktionen zum Suchen, Einfügen, Löschen, Ändern und Auswerten des Datenbestandes der Datenbank bereit.

Das Datenbankmanagementsystem sorgt für die Speicherung der Daten und verwaltet diese in der Datenbank so, dass die interne Verwaltungsstruktur der Daten nach außen nicht sichtbar wird. Die interne Datenstruktur in der Datenbank ist unabhängig von der Programmlogik der nutzenden Anwendungsprogramme (**physische Datenunabhängigkeit**). Änderungen in der internen Datenstruktur wirken damit nicht auf die zugreifenden Anwendungsprogramme. Dem Benutzer bzw. dem Anwendungsprogramm bleiben Details der Speicher- und Zugriffsstrukturen im Datenbanksystem bewusst verborgen.

Die Strukturierung und Darstellung der Daten in der Datenbank erfolgt auf Basis eines Datenmodells. An der Logik des Datenmodelles orientiert, verfolgt das DBMS eine entsprechende Denkweise in der Speicherung und Verwaltung der Daten. Deshalb spricht man von einem **relationalen Datenbanksystem**, wenn das dem Datenbanksystem zugrundeliegende Datenmodell nach dem relationalen Schema (relationales Datenmodell) aufgebaut ist. Das DBMS bietet damit alle notwendigen Funktionen, die für den Umgang und die interne Verwaltung von Daten in einer relationalen Datenstruktur, also einem Tabellenschema, notwendig sind. Es liegt damit eine **relationales Datenbankmanagementsystem** (RDBMS) vor.

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:42

Das Datenbankmanagementsystem übt die zentrale Kontrolle über die Zugriffe der Anwendungsprogramme auf die Datenbank aus und koordiniert die Zugriffe mehrerer Anwendungen. Die Zugriffe der Anwendungsprogramme auf die Daten der Datenbank müssen durch das DBMS in einer wohldefinierten Weise verwaltet und verarbeitet werden, damit die Datenbank stets in einem konsistenten Zustand bleibt. Dem DBMS kommt damit eine sehr wichtige Aufgabe zu, die über die reine Funktionalität zur Datenspeicherung und Datenwiedergewinnung weit hinausgeht.

Um im Fehlerfall einem Datenverlust vorzubeugen und eine hohe Verfügbarkeit der Daten zu gewährleisten, arbeitet das DBMS mit Sicherungs- und Wiederherstellungsmechanismen. Zur Gewährleistung des Datenschutzes stellt das DBMS ein Benutzer- und Zugriffskonzept zur Verfügung. Somit können den verschiedenen Datenbankbenutzern unterschiedliche Rechte zugeteilt werden, was in einem typischen und häufig anzutreffenden Mehrbenutzerbetrieb unumgänglich ist. Gleichzeitig bietet das Datenbankmanagementsystem Konzepte und Komponenten zur Überwachung der Konsistenz und Integrität der Daten. Bei der simultanen Nutzung von Daten koordiniert und synchronisiert es die Zugriffe der verschiedenen Anwendungsprogramme.

2.2.4 Datenbanksprachen

Datenbanksprachen dienen der Kommunikation zwischen Benutzer bzw. Anwendungsprogramm und dem Datenbanksystem. Mit Hilfe einer Datenbanksprache können Benutzer und Datenbankanwendung die gewünschten Datenbankoperationen an das Datenbanksystem übermitteln. Das entsprechende Ergebnis einer Datenbankoperation wird über den Kommunikationskanal zwischen Datenbankmanagementsystem und der Anwendung bereitgestellt.

Das Datenbankmanagementsystem nutzt ebenfalls Datenbanksprachen zur Erzeugung, Nutzung und Bearbeitung der Datenbanken bzw. der Daten in den Datenbanken. Die Datenbanksprachen des DBMS sind standardisiert, dennoch häufig für das eigene System gezielt optimiert. Allgemein können die Inhalte bzw. Operationen der Datenbanksprachen in fünf verschiedene Kategorien unterteilt werden:

- **Abfragesprache** (engl.: ***Q**uery **L**anguage*, kurz: *QL*): Ermöglicht die beschreibende Abfrage und Wiedergewinnung von Daten aus der Datenbank. Die Abfrageoperation benötigt keine detaillierten Kenntnisse über den Aufbau der Datenbank. Diese Kategorie wird in Kapitel 6 mit dem SELECT-Befehl der Abfragesprache SQL ausführlicher behandelt.

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:42

- **Datendefinitionssprache** (engl.: ***D**ata **D**efinition **L**anguage*, kurz: *DDL*): Ermöglicht die Erstellung und Bearbeitung der Struktur der Datenbank. Mit ihr lassen sich Objekttypen und Integritätsregeln, die in der Datenbank zum Einsatz kommen, definieren und deklarieren. Sie unterstützt die Implementierung des logischen Datenmodells.
- **Datenmanipulationssprache** (engl.: ***D**ata **M**anipulation **L**anguage*, kurz: *DML*): Ermöglicht Operationen zum Hinzufügen, Auslesen, Ändern und Löschen der in der Datenbank enthaltenen Daten.
- **Speicherbeschreibungssprache** (engl.: ***D**ata **S**torage **D**escription **L**anguage*, *DSDL*): Diese Kategorie übernimmt die Beschreibung der physischen Datenorganisation innerhalb eines Datenbanksystems.
- **Datenüberwachungssprache** (engl.: ***D**ata **C**ontrol **L**anguage*, kurz: *DCL*): Ermöglicht die Festlegung von Zugriffsrechten und der Konzepte für die Benutzerkontrolle.

2.2.5 Datenbankschnittstellen

Ein Datenbankmanagementsystem bietet für den Zugriff von Anwendungsprogrammen auf die Datenbank und die Kommunikation mit dem DBMS spezialisierte und standardisierte Schnittstellen an. Über **standardisierte Schnittstellen** wird für die Benutzer und Datenbankanwendungen eine vom Datenbanksystem unabhängige Anbindungs- und Kommunikationsmöglichkeit realisiert. Standardisierte Schnittstellen ermöglichen die Verwendung von standardisierten Datenbanksprachen. Die Datenbanksprache SQL (Structured Query Language) ist ein häufig angewandter Standard einer Datenbanksprache in relationalen Datenbanksystemen (▶ Kap. 3 und 6).

Die programmtechnische Implementierung der Datenbankschnittstelle wird als **Middleware** bezeichnet. Die Middleware übersetzt zwischen den in einer standardisierten Datenbanksprache übergebenen Befehlen und den konkreten Datenbankbefehlen des Datenbankmanagementsystems zur Durchführung der Datenbankoperation auf der Datenbank (▶ Abb. 2.6).

Spezialisierte Schnittstellen haben oft den Vorteil, dass sie für eine höhere Geschwindigkeit beim Datenbankzugriff optimiert sind. Dafür müssen aber auch die spezifischen Datenbanksprachen des jeweiligen Datenbankmanagementsystems genutzt werden, wodurch die Unabhängigkeit zwischen den betrieblichen Anwendungsprogrammen und dem verwendeten Datenbanksystem verloren geht.

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:42

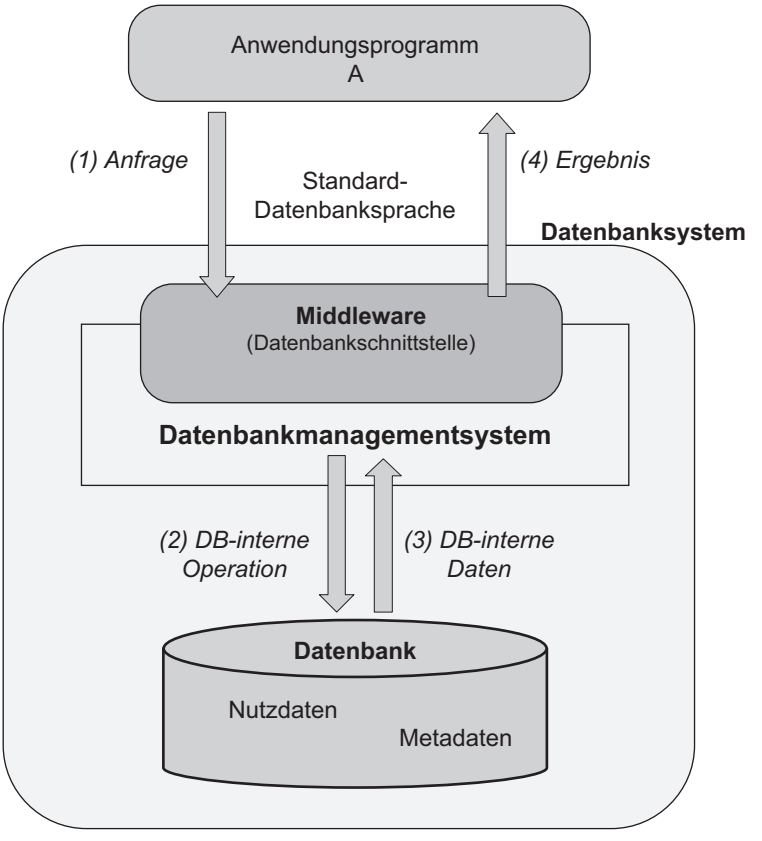


Abb. 2.6: Die Datenbanksprache als Schnittstelle zwischen Datenbanksystem und Anwendung

2.3 Eigenschaften und Aufgaben von Datenbanksystemen

Einem Datenbanksystem obliegt eine Vielfalt von Aufgaben, die für die Verwendung im betrieblichen Einsatz von Bedeutung sind. Ohne den Anspruch auf Vollständigkeit in der Aufzählung zu erheben, sind die wichtigsten Aufgaben und Eigenschaften in folgenden Punkten zu sehen:

- **Persistenz** der Daten
  Die Datenobjekte werden **dauerhaft**, über die Laufzeit des erzeugenden Anwendungsprogramms hinaus, gespeichert und stehen damit zum Start einer neuen Anwendung wieder zur Verfügung.

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:42

- **Datenintegrität** und Einhaltung von **Integritätsbedingungen**
  Das DBS besitzt Mechanismen, die zur Wahrung der Datenkonsistenz bei der Ausführung von Datenbankoperationen dienen. Das System muss dafür sorgen, dass die Daten in sich widerspruchsfrei und korrekt bleiben. Dies bedingt insbesondere Transaktions- und Synchronisationskonzepte. Mechanismen für die Datenintegrität können Eingabekontrollen von zulässigen Wertetypen und Wertebereichen oder die Kontrolle von Integritätsbedingungen sein.
- **Zentrale Datenspeicherung und Datenkontrolle**
  Das DBMS übt die zentrale Kontrolle über eine von vielen Benutzern und Anwendungsprogrammen verwendete Datenmenge aus. Der Zugriff auf die Datenbank kann nur über das DBMS erfolgen.
  Die Aufgabe der Datenspeicherung und Organisation wird zentral gelöst und von den speziellen Auswertungen und Nutzungen der Daten für die fachlichen Aufgabenstellungen entkoppelt.
- **Mehrbenutzerbetrieb**
  Das DBMS übt die Zugriffskontrolle und die Koordination für den gleichzeitigen Zugriff mehrerer Benutzer und Anwendungsprogramme auf das gleiche Datenobjekt aus. Hierzu sind Sperrmechanismen und Synchronisationsmechanismen notwendig.
- **Datenzugriffskontrolle**
  Bereitstellung von Zugriffs- und Berechtigungskonzepten für den Ausschluss unautorisierter Zugriffe.
- **Logische** und **physische Datenunabhängigkeit**
  Aus Sicht der Anwendungsprogramme ist die physische Struktur und Speicherung der Daten in der Datenbank nicht relevant, d. h. die physische Datenorganisation kann verändert werden, ohne dass die Anwendungsprogramme geändert werden müssen. Beispielsweise kann aus Speichereffizienzgründen die Partitionierung einer Tabelle verändert werden, ohne Auswirkung auf die Anwendungsprogramme zu haben.
  Die logische Datenunabhängigkeit verkörpert die Möglichkeit, bestehende Strukturen logisch verändern zu können, ohne die zugehörigen Anwendungsprogramme ändern zu müssen. Beispielsweise können neue Tabellen und Beziehungen für die Speicherung neuer Datenobjekte in die Datenbank aufgenommen werden, diese also damit erweitert werden.
- Bereitstellung von **Datenbankoperationen**
  Für den Aufbau von Datenstrukturen und die Verarbeitung von Daten muss das DBMS entsprechende Datenbankoperationen bzw. Funktionen zur Verfügung stellen. Meist wird eine entsprechende **Datenbanksprache** mit einer **Schnittstelle** für die Ausübung der Datenbankoperationen zur Verfügung gestellt.

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:42

- Gewährleistung der **Sicherheit vor Verlust und Beschädigung** von Daten
  Dies beinhaltet für das DBMS Mechanismen, die den Verlust von Daten bei Systemausfällen und Konflikten vermeiden, ebenso wie die Erstellung von Sicherungen und die Wiederherstellung (engl.: *Recovery*) von korrekten Datenbeständen im Fehlerfall.
- Bereitstellung verschiedener **Schnittstellen** für den Zugang
  Für die Benutzer und Anwendungsprogramme sowie für die Systemadministration muss ein Zugang zu den verfügbaren Datenbankoperationen des DBS vorhanden sein. Der Zugang wird über sogenannte Datenbankschnittstellen ermöglicht, die in unterschiedlicher Form angeboten werden, beispielsweise als grafische Benutzeroberfläche oder als Kommandoeditor für die Anwendung einer Datenbanksprache wie SQL.
- Regelung und **Synchronisation** bei parallelem Datenzugriff
  Das DBMS muss Mechanismen bereitstellen, die eine Behebung von Konflikten und eine Regelung des gemeinsamen Zugriffs auf identische Datenobjekte ermöglichen. Die Erhaltung der Datenkonsistenz muss jederzeit gewährleistet sein.
- **Transaktionskonzepte**
  Eine Transaktion ist die Zusammenfassung mehrerer einzelner Datenbankoperationen zu einer logischen Einheit, die entweder vollständig oder überhaupt nicht ausgeführt wird. Dies dient zur Absicherung der Datenkonsistenz im Fehlerfall bzw. bei Systemausfall.

2.4 Architektur von Datenbanksystemen

2.4.1 Übersicht

Für ein besseres Verständnis der grundlegenden Komponenten und deren Zusammenwirken in der Funktionsweise eines Datenbanksystems werden **Architekturmodelle** zur Erklärung eingesetzt.

Die Architektur eines Datenbanksystems kann auf unterschiedliche Art betrachtet werden:

- Die **Schema-Architektur** beschreibt den Zusammenhang zwischen den verschiedenen Schichten, in denen die **Daten einer Datenbank** repräsentiert werden und den Datenbankanwendungen.

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:42

- Die **System-Architektur** beschreibt den **Aufbau eines Datenbankmanagementsystems** und seiner Komponenten, Bausteine und Werkzeuge. Die Schnittstellen zwischen den Komponenten sind standardisiert.
- Die **Anwendungsarchitektur** beschreibt Benutzerkomponenten und Schnittstellen und die Einbindung des Datenbanksystems in eine Datenbankanwendung, typischerweise auf Basis eines Client-Server-Modells (▶ Kap. 2.4.3).

2.4.2 Schema-Architektur

Die Daten in einem Datenbanksystem werden auf unterschiedlichen Abstraktionsebenen repräsentiert. Die Schema- oder auch Schichtenarchitektur fokussiert auf die Differenzierung zwischen dem fachlogischen Inhalt einer Datenbank, der Beschreibung der Struktur einer Datenbank und der Repräsentation der Daten in der Verwendung nach außen.

Das **Drei-Schichten-Schemaarchitektur-Modell** wurde 1975 vom *Standards Planning and Requirements Committee* (**SPARC**) des *American National Standards Institute* (**ANSI**) entwickelt und beschreibt die Trennung von Datenrepräsentation bzw. der Datenbankanwendungen und der physikalischen Datenhaltung durch drei Ebenen (▶ Abb. 2.7):

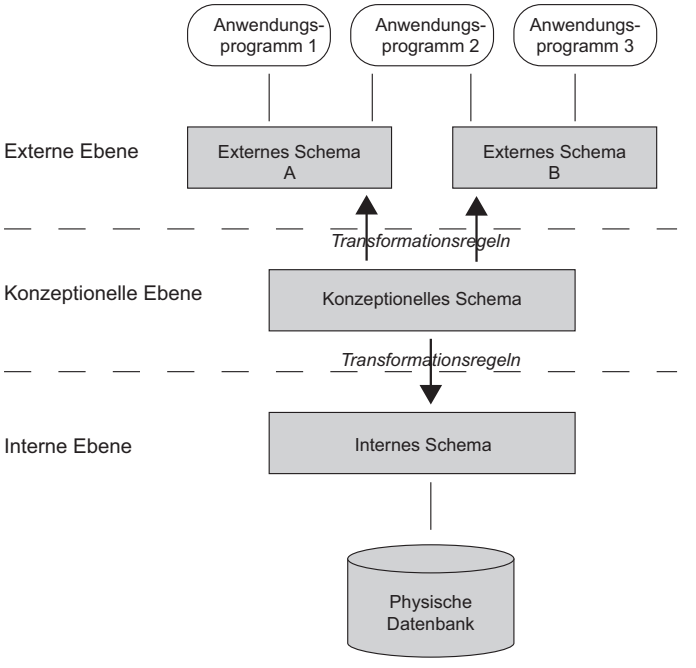


Abb. 2.7: Drei-Schichten-Modell der Daten im Datenbanksystem

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:42

- **Interne Ebene**: Beschreibt die Struktur der **physischen Speicherung** der Daten.
- **Konzeptionelle Ebene**: Beschreibt die Struktur und die sachlogischen Zusammenhänge der Daten aus **fachlicher Sicht**.
- **Externe Ebene**: Beschreibt die Daten aus **anwendungslogischer Sicht**. Benutzer und Datenbankanwendungen greifen nicht direkt auf die Daten der Datenbank zu, sondern arbeiten mit der externen Ebene und sind dadurch vor Auswirkungen durch Änderungen in der Datenbankstruktur geschützt.

Zwischen den verschiedenen Ebenen werden Abbildungen bzw. Transformationsregeln definiert, durch die Anfragen und Änderungen sowie Datendarstellungen zwischen den einzelnen Ebenen übersetzt werden. Die Transformation wird auf Basis von definierten Regeln durchgeführt und findet in zwei Richtungen statt:

- Anfragen bzw. Zugriffswünsche der externen Ebene werden in Form von Abfrageoperationen aus der konzeptionellen Ebene transformiert.
- Die Datenstrukturen der konzeptionellen Ebene werden nach den Gesetzmäßigkeiten des verwendeten Datenbankmodells in physische Speicherstrukturen der internen Ebene transformiert.

Die drei Ebenen repräsentieren verschiedene Abstraktionsebenen im Hinblick auf die Daten. Für jede dieser Ebenen sind in der Zuständigkeit im Unternehmen diverse Rollen vorgesehen, die in der Erarbeitung und Weiterentwicklung auch unterschiedliche Zielsetzungen verfolgen. In Abbildung 2.8 werden die den drei Ebenen zugeordneten Rollen veranschaulicht.

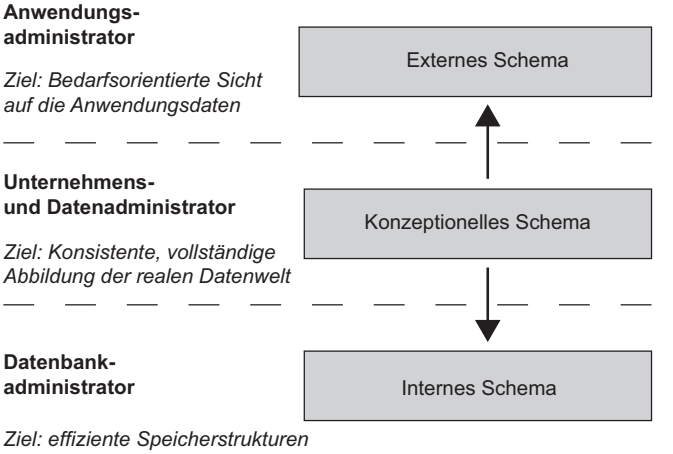


Abb. 2.8: Rollen und Ziele im Drei-Schichten-Modell

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:42

Konzeptionelle Ebene

Die konzeptionelle Ebene wird auch logische Ebene genannt. Sie beschreibt in ihrer sachlogischen Gesamtheit durch ein **konzeptionelles Datenmodell** die Gesamtschau derjenigen Daten, die in der Datenbank verwaltet werden. In der Phase des Datenbankenwurfs und der Datenmodellierung wird dieses konzeptionelle Datenmodell entwickelt.

Die konzeptionelle Ebene definiert ein Schema für die Struktur der gesamten Datenbank. Das Schema enthält alle aus fachlicher Sicht relevanten Datenobjekte und ihre sachlogischen Beziehungen. Darüber hinaus werden Integritäts- und Zugriffsbedingungen, unabhängig von der tatsächlichen Implementierung in der internen Ebene, definiert.

Die konzeptionelle Ebene kann als Referenz und Ausgangspunkt für die interne und externe Ebene gesehen werden. Die externe Ebene agiert in logischer Hinsicht nur mit der konzeptionellen Ebene, nicht jedoch direkt mit der internen Ebene.

Bei der Entwicklung des konzeptionellen Schemas ist Datenneutralität hinsichtlich des Anwendungsprogramms zu wahren, d. h. es darf keine Festlegung der Datenstruktur zugunsten einer Anwendung erfolgen. Auf der anderen Seite ist die Datenunabhängigkeit hinsichtlich der physischen Speicherung bzw. der physischen Datenorganisation zu wahren. Die Zugriffsstruktur und Speicherstruktur darf keinen Einfluss auf die Datenstruktur im konzeptionellen Schema nehmen.

Die Erarbeitung eines konzeptionellen Schemas liegt in der Zuständigkeit des **Unternehmens- bzw. Datenadministrators**. Ein Beispiel für ein konzeptionelles Schema kann das in Abbildung 2.9 dargestellte Entity-Relationship-Modell sein.

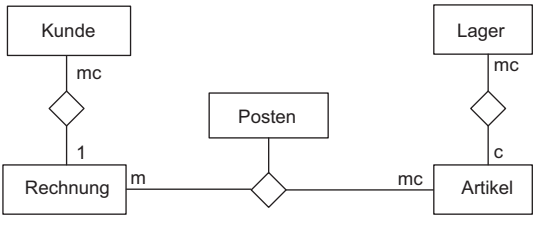


Abb. 2.9: Beispiel konzeptionelles Datenmodell

Externe Ebene

Die externe Ebene repräsentiert die Daten und realisiert den Zugriff auf die Daten aus Sicht der Datenbankanwendungen und Benutzer. Jede Benutzerdomäne beschreibt ihre eigene, der jeweiligen Aufgabe angepasste Sicht auf die in der Datenbank gespeicherten Daten des Unternehmens.

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:42

Es können mehrere externe Schemata gleichzeitig existieren und jedes einzelne stellt einen für den Benutzer oder die Anwendung spezifisch aufbereiteten Ausschnitt aus dem gesamten Datenbestand der Datenbank dar.

Da ausschließlich diejenigen Daten einsehbar sind, die in einem externen Schema definiert wurden, wird der Benutzer von der Komplexität der Datenbank abgeschirmt und der Umgang mit der Datenbank vereinfacht. Aufgrund des wohldefinierten und bedarfsorientierten Datenzugriffs entsteht zusätzlich ein Zugriffsschutz.

Durch die Definition eines **externen Schemas** können die jeweiligen Datentypen und Datenstrukturen der Datenbankanwendung auf diejenigen des Datenbanksystems abgebildet werden. Externe Schemata können auch hierarchisch aufgebaut sein, d. h. ein externes Schema kann von einem anderen oder mehreren externen Schemata abgeleitet sein.

Ein externes Schema muss mittels kontrollierter Transformationen, beispielsweise mittels eines SQL-Befehls, aus dem konzeptionellen Schema abgeleitet werden können. Die Ausprägung eines externen Schemas kann als immaterielle Sicht (*View*) gegeben sein.

Für die Erarbeitung und Definition der externen Schemata ist der **Anwendungsadministrator** zuständig. Beispiel für ein externes Schema kann die Transformation einer virtuellen Tabelle bzw. eines *Views* sein (▶ Abb. 2.10).

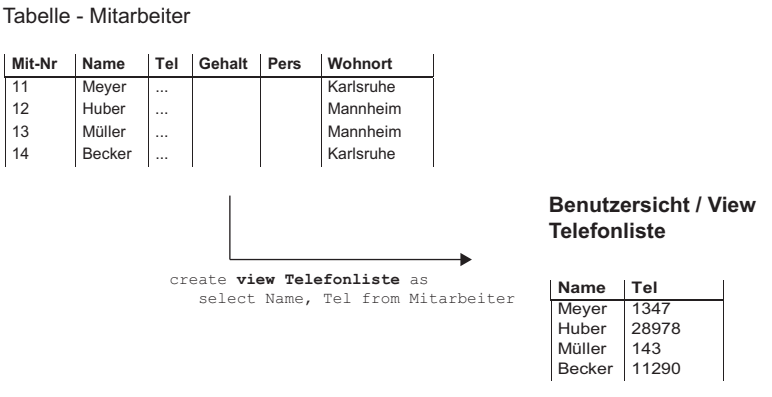


Abb. 2.10: Beispiel einer virtuellen Tabelle als externes Schema

Interne Ebene

Die interne Ebene wird auch physische Ebene genannt. Sie liegt dem physischen Speicher, der eigentlichen Datenbasis, am nächsten, ist aber nicht mit diesem gleichzusetzen. Auf der internen Ebene ist die **physische Datenorganisation** definiert.

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:42

Die interne Ebene enthält alle Informationen der physischen Speicherung und Organisation von Daten auf einem Datenträger. Neben der Beschreibung der Details der physischen Datenspeicherung in der Art und im Aufbau der Datenstrukturen beschreibt das **interne Schema** die speziellen Zugriffspfade auf die physischen Datensätze, also wie auf die Daten zuzugreifen ist.

Auf der konzeptionellen Ebene wird von der internen Ebene abstrahiert. Die für den Benutzer irrelevanten Details der physischen Datenspeicherung bleiben damit der Anwendung verborgen. Die Zuständigkeit für das interne Modell liegt beim **Datenbankadministrator**. Ein Beispiel für ein internes Schema zeigt Abbildung 2.11.

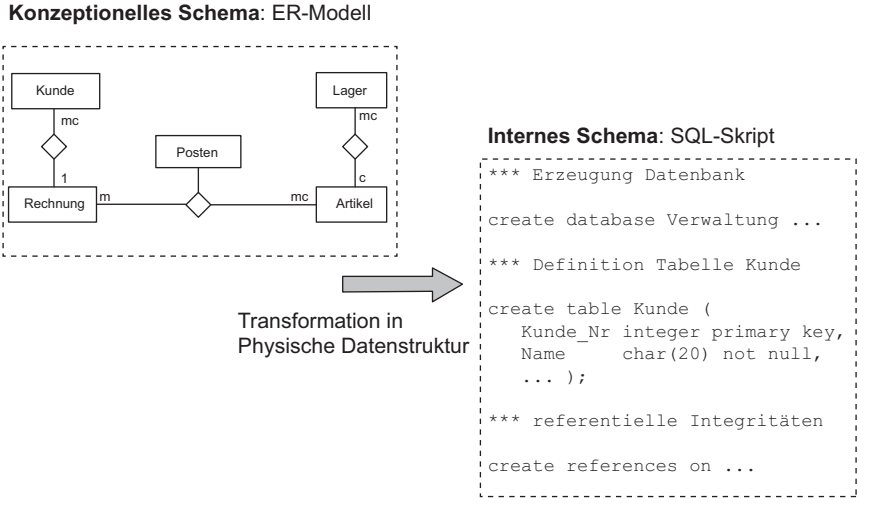


Abb. 2.11: Beispiel für internes Schema zur physischen Datenorganisation

2.4.3 Anwendungsarchitektur

Ausgangssituation

Die Anwendungen und Programme, die auf die Daten eines Datenbanksystems zugreifen, nennt man **Datenbankanwendungen**. Hierbei kann es sich um Anwendungen handeln, die speziell für die Bearbeitung und das Auswerten von Daten in einer Datenbank entwickelt wurden und solchen Anwendungen ähneln, die eine Datenbank selbst zur Verwaltung der Daten benötigt. Beispielsweise verwenden ERP-Systeme (▶ Abb. 1.1.6) für ihre Datenverwaltung typischerweise Datenbanksysteme und sind als Datenbankanwendungen oder als **datenbankbasierte Informationssysteme** zu interpretieren.

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:42

Für die Nutzung von Datenbanksystemen im betrieblichen Umfeld stehen verschiedene **technologische Varianten** zur Verfügung. Die Anforderungen an eine technologische Lösung in der Installation eines Datenbanksystems resultieren aus unterschiedlichen Betrachtungen, wie beispielsweise der Anzahl an Anwendern, der verfügbaren Netzwerktechnologie, der notwendigen Performance oder der zu berücksichtigenden Sicherheitsaspekten. Die Bewertung einer technologischen Architektur nach Vor- und Nachteilen ist immer kontextabhängig, d. h. orientiert sich zunächst an den gestellten Anforderungen. Es gibt keine Variante, die in allen Belangen vorteilhaft ist.

Für eine Erläuterung der Prinzipien in der Funktionsweise der technologischen Architektur wird zunächst ein **Datenbank-Architekturmodell** betrachtet, welches das Gesamtsystem in ein **Frontend** und ein **Backend** untergliedert. Frontend und Backend sind eigenständige Prozesse bzw. lauffähige Software-Komponenten, die bestimmte Aufgaben übernehmen und miteinander in Kommunikation stehen. Ein Frontend repräsentiert den Verarbeitungsprozess einer Datenbankanwendung, während das Backend die Verarbeitung der Operationen in der Datenbank widerspiegelt. Die Architektur von Datenbankanwendungen und Datenbanksystemen basiert typischerweise auf einem in Abbildung 2.12 dargestellten Modell.

Die Abbildung 2.12 ist im Hinblick auf die Funktionsweise folgendermaßen zu verstehen:

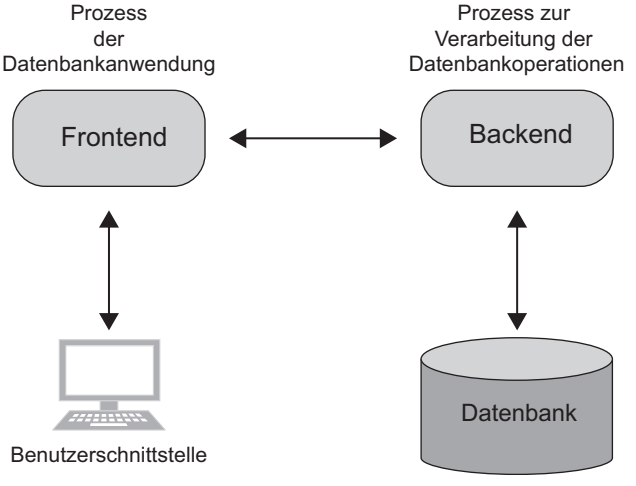


Abb. 2.12: Architekturmodell Frontend und Backend

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:42

- Die beiden Komponenten *Frontend* und *Backend* sind eigenständige Prozesse mit gegenseitiger Kommunikation.
- Der Frontend-Prozess kommuniziert mit dem Benutzer; ist also nach vorne zur Anwendung hin gerichtet.
- Der Backend-Prozess kommuniziert mit der Datenbank; ist also im Hintergrund für die Zusammenarbeit mit dem Datenbanksystem zuständig.
- Die beiden Prozesse *Frontend* und *Backend* können auf demselben oder auf verschiedenen Rechnern installiert sein.
- Falls *Frontend* und *Backend* auf unterschiedlichen Rechnern installiert sind und dort laufen, werden diese als **Client** (Frontend) und **Server** (Backend) bezeichnet.

Im Weiteren werden nun diverse Installationsvarianten von Datenbanksystemen betrachtet. Die Installationsvarianten unterscheiden sich maßgeblich darin, auf welchen Rechnern bzw. Plattformen die beiden Komponenten von Frontend und Backend installiert sind. In beiden nachfolgend vorgestellten Varianten sind auf einem Unternehmensnetzwerk die Arbeitsstationen der Benutzer sowie auch ein zentraler Rechner im Netzwerk als Server verfügbar.

File-Server-Prinzip

In einer ersten Variante werden herkömmliche Datenbanksysteme betrachtet, die im lokalen Netzwerk (LAN) ihre Dienste anbieten. In der nachfolgenden Betrachtung spielt die Datenübertragung zwischen dem Server und der Arbeitsstation eine wichtige Rolle. Auf den Arbeitsstationen im LAN sind jeweils das Datenbanksystem sowie das datenbankbasierte Anwendungssystem installiert. Im LAN existiert ein **Datei-Server**, in der Begrifflichkeit als **File-Server** beschrieben, der für die Arbeitsstationen zur Ablage von Dateien verfügbar ist. Alle Anwender haben damit Zugriff auf die gleichen Daten. Eine Mehrfachspeicherung der Daten auf den lokalen Arbeitsstationen entfällt damit.

Bezogen auf das Architekturmodell in Abbildung 2.12 sind Frontend und Backend gemeinsam auf jeder Arbeitsstation installiert. Die Komponente Datenbank ist auf dem Server im LAN abgelegt. Die Funktionsweise eines Datenbanksystems im File-Server-Betrieb soll anhand des nachfolgenden Beispiels erläutert werden:

Beispiel:
Auf dem Datei-Server im LAN befindet sich eine Tabelle mit 10.000 Kundenadressen. Ein Anwender am Datenbanksystem möchte die An-

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:42

schrift des Kunden mit Nr. 12345 ermitteln. Für die Bearbeitung dieser Anfrage am Datenbanksystem der Arbeitsstation muss zunächst die komplette Tabelle *Kunde* über das LAN zur lokalen Arbeitsstation transferiert werden.

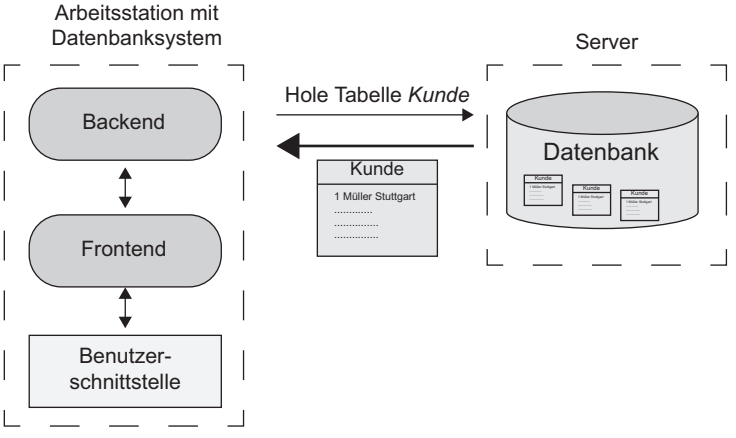


Abb. 2.13: Funktionsweise im File-Server-Betrieb

Nach Ende der Bearbeitung der Kundentabelle wird das Datenbanksystem an der Arbeitsstation die vollständige Tabelle zurück auf den Datei-Server transferieren und speichern.

Die mit einem File-Server-Betrieb gegebenen Probleme, insbesondere in einem Mehrbenutzerbetrieb, sind offensichtlich:

- Die Zugriffsrechte auf die im File-Server abgelegten Dateien sind nur durch das zugrundeliegende Betriebssystem kontrollierbar. Eine benutzerspezifische Zugriffsregelung nach der Datenbank-Anwendung ist nur an der Arbeitsstation, aber nicht auf dem Server möglich.
- Der File-Server kann keine Koordination nebenläufiger Zugriffe aus unterschiedlichen Arbeitsstationen vornehmen. Damit kann die Gewährung der Datenkonsistenz nicht aufrechterhalten werden. Beispielsweise kann der Server nicht kontrollieren, welche gleichzeitigen Änderungen in der Kundentabelle beim Zurückspeichern Gültigkeit haben sollen.

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:42

Client-Server-Prinzip

Bei einem **Client-Server-Prinzip** ist die gesamte Datenbanklogik durch die beiden Prozesse Client und Server verteilt installiert. Die beiden Prozesse sind auf die Arbeitsstationen und auf den Server aufgeteilt. Auf der Arbeitsstation ist nicht das komplette Datenbanksystem installiert, sondern lediglich das Frontend als Teil des Datenbanksystems. Auf dem Server wird das Backend einmalig installiert. Der Server als zentraler Rechner im LAN wird als **Datenbank-Server** nun komplett die Verwaltung der Datenbank und Koordination der Zugriffe aller Anwender übernehmen.

Bei einer Client-Server-Architektur wird ein optimales Gleichgewicht zwischen dem Datenbank-Server und den Arbeitsplatzstationen hinsichtlich der Datenstrommenge sowie der Rechnerleistung (Performance) bei Client und Server angestrebt. Intention ist die optimale Verteilung von Arbeitsprozessen zwischen den lokalen Arbeitsstationen und der zentralen Server-Einheit im LAN.

Die Arbeitsstation mit der Datenbankanwendung als Client erledigt alle Ein- und Ausgabeaktivitäten und ermöglicht den effizienten Einsatz einer grafischen Benutzeroberfläche, einem sogenannten GUI (*Graphical User Interface*).

Die Verarbeitung von Datenbank-Operationen, beispielsweise einer Datenabfrage, findet dort statt, wo der zentrale Datenbestand lagert, also auf dem Server-Rechner. Damit wird die Datenstromkapazität zwischen Frontend und Backend deutlich reduziert.

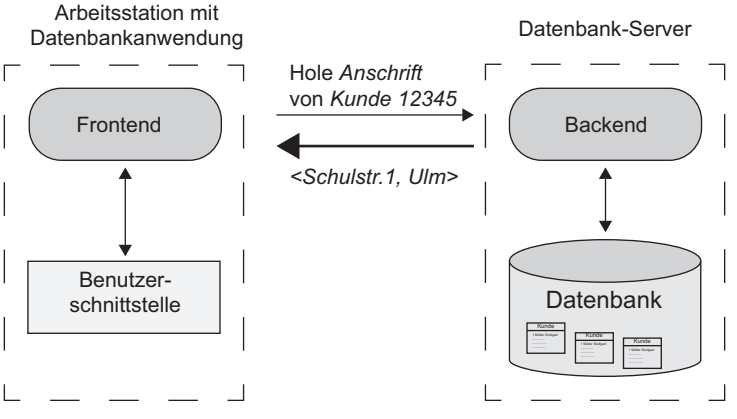


Abb. 2.14: Funktionsweise im Client-Server-Betrieb

Das eigentliche Datenbanksystem und damit die Datenbank-Operationen wie beispielsweise das Ändern eines Datensatzes, laufen auf dem Server-Rechner. Dies erfordert eine hohe Leistungsfähigkeit auf dem Server-

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:42

Rechner. Im Vergleich zu einem File-Server-Betrieb wird die Leistungsfähigkeit von den Arbeitsstationen hin zum Server verlagert.

Das Frontend der Datenbankanwendung läuft vollständig auf der lokalen Arbeitsplatzstation (Client). Nur wenn neue Daten benötigt werden, erfolgt eine Übertragung einer Nachricht an den Server; meist mittels einer SQL-Anweisung. Der Datenbank-Server (Backend) sucht die benötigten Daten aus dem zentralen Datenbestand und sendet das Ergebnis der SQL-Anfrage über das LAN zurück an den Client.

Die **Vorteile** in der Client-Server-Konstellation sind vielfältig:

- Die Anforderungen an die Leistungsfähigkeit des Gesamtsystems können zwischen Arbeitsstation und Server besser umgesetzt werden.
- Eine größere Flexibilität im Einsatz von Softwareprodukten sowie der Integration verschiedener Rechnerwelten ist gegeben. Ebenso lässt sich diese Architektur leichter skalieren und damit der Unternehmensexpansion schneller anpassen.
- Die Administration für das Datenbanksystem insgesamt wird vereinfacht.
- Sicherheitsaspekte sind leichter umsetzbar, weil das Serverumfeld auf einem einzigen Rechner systemtechnisch besser kontrollierbar bleibt.

Wesentlicher **Nachteil** in der Client-Server-Welt bleibt das Risiko, dass bei einem Ausfall des Servers oder des LAN-Betriebs die Arbeitsstationen nicht mehr funktionsfähig sind.

2.5 Klassifikation von Datenbanksystemen

2.5.1 Orientierung am Datenmodell

Eine Gegenüberstellung von Datenbanksystemen lässt sich nach verschiedenen Merkmalen vornehmen. Neben der Frage der technologischen Ausprägung eines Datenbanksystems, beispielsweise eines Client-Server-Systems, spielt vor allem das zugrundeliegende **Datenmodell** für den Umgang und die Speicherung von Daten ein wichtiges Unterscheidungsmerkmal. Hier wird nun die Klassifikation von Datenbanksystemen nach dem zugrundeliegenden Datenmodell betrachtet.

Eine Datenbank stellt einen relevanten und vereinfachten Ausschnitt der realen Welt dar. Das Datenbankmanagementsystem verwaltet die Datenbank. Dabei gibt es verschiedene Konzepte, wie Daten strukturell organisiert und verwaltet werden.

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:42

Ein **Datenmodell** ist eine abstrakte Beschreibung darüber, welche Daten in welchem logischen Zusammenhang gegeben sind. Es beschreibt die Struktur der Daten und ihre Beziehungen zueinander.

Folgende Komponenten sind für ein Datenmodell strukturgebend:

- **Objekttypen**, z. B. *Kunde* oder *Rechnung*, und **Beziehungstypen**, z. B. *Rechnung gehört Kunde*. Diese Komponenten definieren **statische Eigenschaften** eines Datenmodells.
- **Operationen**, z. B. *Erstellung* und *Pflege* von Kundendaten, und Beziehungen zwischen den Operationen. Das sind **dynamische Eigenschaften** eines Datenmodells.
- **Integritätsbedingungen** definieren Regeln zur Erhaltung der Datenintegrität und Datenkonsistenz, z. B. *jede Rechnung muss zu genau einem Kunden gehören*.

Datenmodelle können je nach Zielsetzung und der eingesetzten Entwurfsphase ihrer Verwendung nach folgenden Arten unterschieden werden:

- **Konzeptionelles Datenmodell**:
  Ein Datenmodell, welches aus der fachlichen Sicht der Problemstellung heraus die relevanten Datenstrukturen beschreibt. Das konzeptionelle Datenmodell ist unabhängig von der späteren technischen Implementierung auf einem Datenbanksystem. Hier werden oftmals sogenannte semantische Datenmodelle wie beispielsweise das Entity-Relationship-Modell oder das Objekt-Datenmodell nach UML eingesetzt.
- **Logisches Datenmodell**:
  Ein Datenmodell, welches für die Beschreibung die Logik sowie mögliche Strukturen und Regeln eines Datenbankmanagementsystems berücksichtigt. Das logische Datenmodell basiert inhaltlich auf dem konzeptionellen Datenmodell und transformiert dieses in eine verständliche Form aus Sicht des zugrundeliegenden Datenbanksystems. Beispiel hierfür ist ein relationales Datenmodell, welches durch relationale Datenbankmanagementsysteme verkörpert wird.
- **Physisches Datenmodell**:
  Eine Abbildung des logischen Datenmodells auf eine physische Organisationsform, z. B. eine Tabellenstruktur oder Indizes, des konkret eingesetzten Datenbankmanagementsystems.

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:42

Die Kombination aus dem logischen Datenmodell und dem dazugehörigen Datenbankmanagementsystem, welches das Konzept des logischen Datenmodells unterstützt, nennt man **Datenbankmodell**. Typische Datenbankmodelle eines Datenbanksystems sind das hierarchische, das relationale, das objektorientierte und das netzwerkförmige Datenbankmodell.

2.5.2 Hierarchisches Datenbankmodell

Das **hierarchische Datenbanksystem** oder Datenbankmodell basiert auf dem hierarchischen Datenmodell. Dieses stellt eine einfache und effiziente Form der Datenstrukturierung dar, wie sie in vielen Problemstellungen vorzufinden ist.

Im hierarchischen Datenmodell werden die Daten als **Baumstruktur** organisiert (▶ Abb. 2.15). Ausgehend von einem Element gibt es Unterelemente, die wiederum eigene Unterelemente besitzen können. Die Elemente nennt man **Knoten**, das erste Element, also das Element ohne Vorgänger, ist der Wurzelknoten. Jeder weitere Knoten besitzt genau einen Vorgänger, Elternknoten genannt, kann aber beliebig viele Nachfolger, Kindknoten genannt, haben. Einen Knoten ohne Nachfolger nennt man **Blatt**, die Verbindungen zwischen den einzelnen Knoten wird **Kante** genannt.

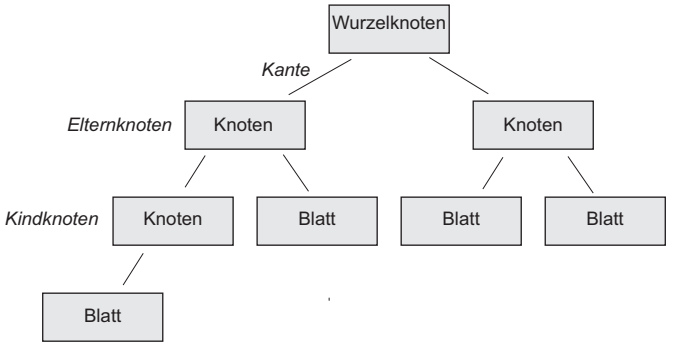


Abb. 2.15: Hierarchische Baumstruktur

Im hierarchischen Datenmodell repräsentiert ein Knoten einen Datensatz. Beziehungen zwischen den einzelnen Datensätzen werden durch die Baumstruktur dargestellt und mit Zeigern realisiert. Besitzen verschiedene Knoten bzw. Datensätze einen oder mehrere identische Nachfolger, hat die hierarchische Baumstruktur den Nachteil, dass die identischen Nachfolger mehrfach vorhanden sein müssen, wodurch Datenredundanz entsteht.

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:42

Die Daten einer hierarchischen Datenbank werden in einer einzelnen sequentiellen Datei gespeichert. Die physische Datenorganisation ist gegenüber Strukturänderungen sehr unflexibel gestaltet. Ein weiterer Nachteil des hierarchischen Datenmodells ist, dass es für komplexe Umgebungen nur schwierig und aufwändig zu modellieren ist. Darüber hinaus sind die Navigationspfade für die Selektion von Daten von der hierarchischen Struktur abhängig und finden ihren Niederschlag in der Programmlogik der Datenbankanwendungen.

Das hierarchische Datenbankmodell ist das älteste Modell und wurde aufgrund seiner Nachteile weiterentwickelt, um insbesondere die Flexibilität in der Modellierung zu erhöhen.

2.5.3 Netzwerk-Datenbankmodell

Das Netzwerk-Datenbanksystem basiert auf dem Netzwerk-Datenmodell und wird als eine Weiterentwicklung des hierarchischen Datenbankmodells gesehen. Daten werden hier ebenfalls in einer Baumstruktur organisiert.

Im Unterschied zum hierarchischen Datenmodell fordert das Netzwerk-Datenmodell keine strenge hierarchische Struktur und erlaubt damit die **Mehrfachverzweigung** von Knoten. Das heißt, ein Kindknoten kann mehrere Elternknoten besitzen. Abbildung 2.16 zeigt den Unterschied zu einer hierarchischen Baumstruktur.

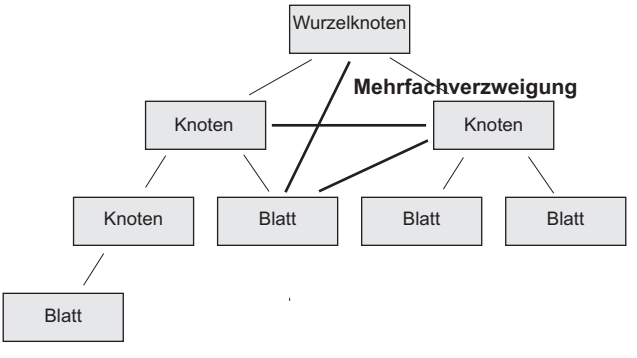


Abb. 2.16: Netzwerk-Struktur

Der Vorteil des Netzwerk-Datenbankmodells im Vergleich zum hierarchischen Datenbankmodell ist, dass durch die Möglichkeit der vernetzten Strukturen weniger Redundanzen entstehen und auch komplexe Objekte und Umgebungen modellierbar sind.

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:42

Die Einfachheit und Übersichtlichkeit geht durch die komplexe Implementierung jedoch verloren. Auch das Datenbankmanagementsystem ist durch die komplexeren Strukturen bei der Verarbeitung der Datenbank langsamer und ineffizienter.

2.5.4 Relationales Datenbankmodell

Das **relationale Datenbankmodell** ist weit verbreitet und bis heute etablierter Standard bei vielen Datenbanksystemen. In der betrieblichen Praxis werden im Umfeld der betriebswirtschaftlichen Kernaufgaben, insbesondere im Umfeld des Einsatzes von Standard-Softwaresystemen, in der Mehrzahl sogenannte **relationale Datenbanksysteme** eingesetzt, die auf diesem Datenbankmodell basieren.

Das Denkmuster für die Organisation der Daten ist hier nach dem relationalen Datenmodell (vgl. Codd 1970) aufgebaut. Zentrales Strukturierungselement des relationalen Datenmodells ist die **Tabelle**. Eine Tabelle im mathematischen Sinne ist durch die Begrifflichkeit einer **Relation** verkörpert.

Im relationalen Datenmodell werden alle Daten ausschließlich in Tabellen und nicht wie im hierarchischen Datenbankmodell sequentiell in einer Datei abgelegt. Durch die Organisation und Strukturierung der Daten in Tabellen sind die Datenstrukturen einfach gehalten und dennoch flexibel handhabbar. Beispielsweise kann im relationalen Modell jederzeit eine Erweiterung mit neuen Tabellen gemacht werden, ohne bisherige Strukturen und Anwendungen damit verändern zu müssen.

Im Vergleich zum Netzwerk- und hierarchischen Datenbankmodell bietet das relationale Datenbankmodell den **Vorteil**, dass es keine strukturellen Abhängigkeiten besitzt. Die Daten müssen nicht abhängig von ihren Beziehungen zueinander abgelegt werden, sondern können in verschiedenen Tabellen unabhängig geordnet werden. Es gibt also keine Navigationspfade, die bei der Bearbeitung von Daten in den Anwendungsprogrammen zu beachten sind. Strukturänderungen können einfach durch die Erzeugung neuer Tabellen realisiert werden. Auch das Löschen oder Hinzufügen von Attributen bzw. Tabellenspalten kann durchgeführt werden, ohne dass dies Auswirkungen auf die interne Logik von Datenbankanwendungen hat.

Eine **Schwachstelle** des relationalen Datenbankmodells liegt in der Zugriffsperformance auf die Daten. Grund hierfür ist, dass Daten nicht wie im hierarchischen Datenmodell sequentiell gelesen werden können, sondern aus verschiedenen Tabellen zusammengeführt werden müssen. Die Performance in der Abarbeitung von Transaktionen auf der relationalen

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:42

Datenbank, oftmals ebenfalls ein Nadelöhr, gilt häufig als Gradmesser für die Leistungsfähigkeit dieser Datenbanksysteme. Transaktionen sind eine feste Folge von Datenbankoperationen, die als eine logische Einheit zu behandeln sind. Beispiel hierfür sind betriebswirtschaftliche Operationen wie Buchungssätze.

Die grundlegenden Eigenschaften im relationalen Datenmodell sowie die Kernelemente von relationalen Datenbanksystemen werden im Kapitel 3 dieses Buches ausführlicher behandelt.

2.5.5 Objektorientiertes Datenbankmodell

Das objektorientierte Datenbankmodell dient der Abbildung und Verarbeitung komplex strukturierter Daten und ermöglicht eine realitätsnahe Modellierung vieler Umgebungen und Objekte.

Objektorientierte Datenbankmodelle fokussieren nicht länger auf die Verwaltung der Daten in Tabellen, sondern auf die Speicherung und Verwaltung der Daten als Objekte im Sinne der Objektorientierung. Dabei umfasst ein Objekt nicht nur die Daten bzw. Attribute, sondern auch die zur Verarbeitung notwendigen Methoden bzw. Operationen des Objekts. Die Zusammenfassung von Daten und Methoden nennt man in der Objektorientierung *Kapselung*. Der Zugriff auf das Objekt erfolgt mit den eigens bereitgestellten Methoden des Objekts.

Zur Identifikation müssen nicht wie beim relationalen Datenmodell Schlüssel definiert werden, sondern jedes Objekt bekommt automatisch eine eindeutige Objektidentität. Die Verwaltung der Objektidentitäten wird durch das Datenbankmanagementsystem übernommen. Weiterhin unterstützt das DBMS den effizienten Zugriff auf die Daten durch Kenntnisse über zusammengehörige Daten.

Objektorientierte Datenbankmodelle bieten weitere typische objektorientierte Konzepte wie die Vererbung oder die Überladung von Methoden. Durch die leistungsfähigeren, objektorientierten Strukturen lassen sich komplex strukturierte Daten, die sich nur schwer in den flach strukturierten Tabellen des relationalen Datenmodells modellieren lassen, besser abbilden.

Die Nachteile eines objektorientierten Datenbankmodells liegen in der fehlenden Standardisierung. Im Gegensatz zu anderen Datenbankmodellen, gibt es für ein objektorientiertes Datenbankmodell kein einheitliches Datenbankmodell.

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:42

2.5.6 Objektrelationales Datenbankmodell

Das **objektrelationale Datenbankmodell** kann als Kombination des objektorientierten und des relationalen Datenbankmodells gesehen werden. Ein relationales Datenmodell wird durch ein objektorientiertes Datenbankmanagementsystem verwaltet und dabei durch Konzepte der Objektorientierung erweitert. Eine Entität des relationalen Datenmodells entspricht dabei einem Objekt.

Die Datenhaltung basiert weiterhin auf Tabellen, jedoch um objektorientierte Konzepte, wie bei der Vererbung bei der Definition von Tabellen, erweitert. Im Unterschied zum relationalen Datenmodell kann nicht nur eine begrenzte Anzahl an möglichen Datentypen verwendet werden, sondern es können darüber hinaus auch benutzerdefinierte Datentypen angelegt werden.

2.5.7 Semistrukturiertes Datenbankmodell

Das **semistrukturierte Datenbankmodell** basiert auf keinem fest vorgeschriebenen Datenmodell. Daten werden nach keiner allgemein gültigen Form strukturiert, sondern tragen die Strukturinformationen mit sich. Sie werden nach dem Prinzip der hierarchischen Baumstruktur aufgebaut, wobei jede Entität unterschiedliche Attribute enthalten kann. Die Speicherung und Verwaltung semistrukturierter Daten bietet vor allem Vorteile für Daten, deren Strukturen unregelmäßig, teilweise oder gänzlich unbekannt sind.

Ein typisches Beispiel für ein semistrukturiertes Datenmodell sind **XML**-Dokumente[4]. In XML-basierenden Datenbanksystemen werden die Daten als hierarchisch geordnete XML-Daten gehalten und können beliebige Strukturen enthalten, solange diese nach XML-Definition »wohlgeformt« sind.

Der Nachteil semistrukturierter Datenbankmodelle ist die im Vergleich zu relationalen Systemen geringere Performance.

4 Die eXtensible Markup Language (erweiterbare Auszeichnungssprache, kurz: XML) hat sich zu einem weit verbreiteten Standard für die Speicherung von Daten etabliert. Das Format ist ein vom WorldWideWeb Consortium (W3C) eingeführter Standard, mit dem sich Daten und deren inhaltliche Struktur nach definierten Regeln abbilden lassen.

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:42

2.6 Entwurf von Datenbanken

Die Nutzbarkeit und Akzeptanz eines Datenbanksystems hängt sehr stark vom Ergebnis des Entwurfes der zugrundeliegenden Datenbank ab. Daher ist dem **Datenbankentwurf** mit einer **konzeptionellen** und **logischen Datenmodellierung** besonderes Augenmerk zu widmen.

Der Datenbankentwurf, oft auch als **Datenbank-Design** bezeichnet, beschreibt die systematische Vorgehensweise zur Entwicklung einer Datenbanklösung und ist durchzuführen, wenn für einen bestimmten Anwendungsbereich ein Datenbanksystem implementiert und eingeführt werden soll. Die Entwicklung von Datenbanksystemen folgt den Prinzipien, Methoden und Konzepten des Software-Engineering, also dem Software-Entwicklungsprozess eines Anwendungssystems.

2.6.1 Life-Cyle-Modell für die Daten

Der Software-Lebenszyklus insgesamt lässt sich in eine Koexistenz eines **Lebenszyklus für die Daten** (*Data-Life-Cycle*) sowie eines **Lebenszyklus für die Anwendung** (*Application-Life-Cycle*) aufteilen (▶ Abb. 2.17). Beide Lebenszyklusmodelle greifen ineinander, sind aber für sich stehende Prozesse.

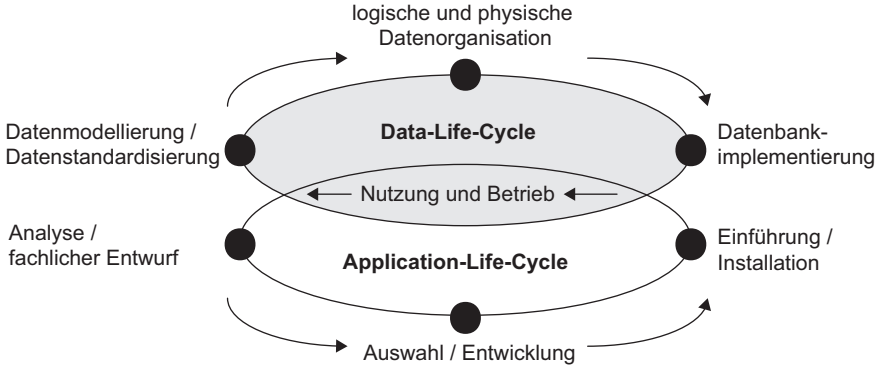


Abb. 2.17: Data- und Application-Life-Cycle

Der **Data-Life-Cycle** ist auf die Analyse, Entwicklung und Einführung eines Datenbanksystems als Grundlage einer betrieblichen Anwendung ausgerichtet. Im Weiteren gilt daher die Betrachtung dem *Data-Life-Cycle* und den damit verbundenen Entwurfsphasen.

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:42

2.6.2 Phasen des Datenbankentwurfs

Phasenmodell als Grundlage

Ein Phasenmodell dient der systematischen Planung, Steuerung und Kontrolle des Entwicklungsprozesses. Es beschäftigt sich von der Beschreibung der Anforderungen bis hin zur Realisierung des Datenbanksystems und ist wichtig, um alle Anforderungen an das Datenbanksystem bei der Entwicklung zu berücksichtigen.

Um einen Ausschnitt aus der realen Welt als digitale Daten in einer anwendungsunabhängigen Form durch ein Datenbanksystem zur Verfügung zu stellen, ist ein stufenweises Vorgehen erforderlich. Der Entwurfsprozess für ein Datenbanksystem vollzieht sich deshalb in mehreren Phasen mit jeweils konkreten Aufgaben. Zur schematischen Darstellung des Entwicklungsprozesses eines Datenbanksystems werden spezifische Phasenmodelle verwendet.

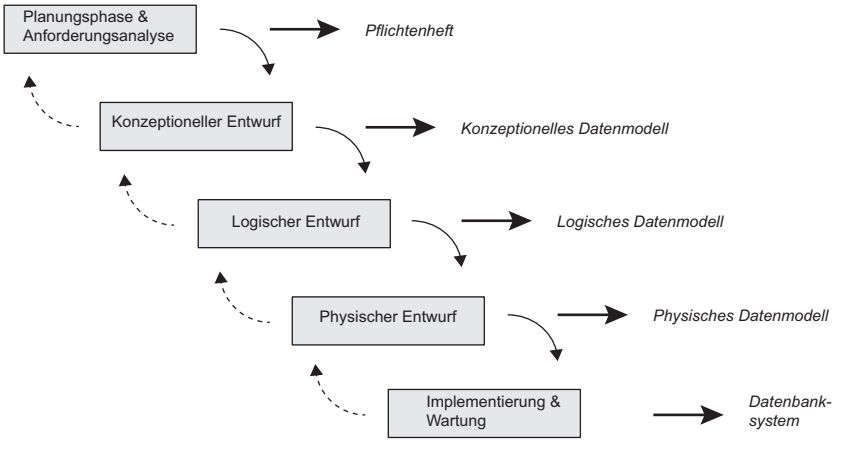


Abb. 2.18: Phasenmodell für den Datenbankentwurf

Das Phasenmodell in Abbildung 2.18 zeigt neben den einzelnen Aufgaben auch die jeweils entstehenden Zwischenergebnisse, sogenannte Artefakte. Jede Phase verwendet zur Lösung der gegebenen Aufgabe spezifische Methoden, beispielsweise den *Entity-Relationship-Ansatz*. Die erstellten Artefakte einer vorausgegangenen Phase dienen als Grundlage für die jeweils nächste Phase des Entwicklungsprozesses.

Das kaskadenartige Phasenmodell zeigt auch Rückverbindungen zur vorhergehenden Phase. Die strikt sequenzielle Abarbeitung jeder einzelnen Phase ist dabei nicht bindend, sondern kann eher als ein optimaler Verlauf des Entwicklungsprozesses gesehen werden. Werden in einer

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:42

Phase aufgrund neuer Erkenntnisse, veränderter Randbedingungen oder neuer Anforderungen, Änderungen vorgenommen, müssen diese in den vorangegangenen Phasen nachvollzogen werden. Demnach kann es notwendig sein, die einzelnen Phasen mehrmals zu durchlaufen; eine **iterative Abarbeitung der Phasen** ist damit gegeben. Auch bei umfangreichen und komplexen Anwendungsbereichen empfiehlt es sich, zunächst nur Teilbereiche systematisch dem Phasenmodell entsprechend abzuarbeiten.

Die Hauptaufgaben im Datenbankentwurf sind die Erstellung eines konzeptionellen Modells, das die aus der fachlichen Sicht notwendigen Daten abbildet, die Überführung dieses Modells in ein logisches Datenbankmodell sowie die Erstellung der externen Schemata in Abhängigkeit der von einer Anwendung bzw. dem Benutzer benötigten Teilmenge der Daten.

Die Entwicklung der verschiedenen Schemata wird in einer durchdachten Reihenfolge ausgeführt. Zuerst wird eine abstrakte Beschreibung der Datenwelt vorgenommen, die dann weiter konkretisiert wird. Die Entwurfsdokumente und Modelle in den frühen Phasen sind systemunabhängig und ermöglichen damit in den nächsten Phasen auch unterschiedliche Implementierungen. Mit der zunehmenden Entwicklungstiefe und der zunehmenden Konkretisierung wird auch die Abhängigkeit zu einem bestimmten Datenbanksystem größer.

Nachfolgende Phasen bauen auf Ergebnissen und Vorgaben der vorangegangenen Phasen entsprechend der Abbildung 2.18 auf.

Anforderungsanalyse und Planungsphase

Die erste Phase beschäftigt sich mit der Analyse der Anforderungen, der Planung des Datenbankentwurfs und des Datenbanksystems. Die Anforderungsanalyse und Planung sollten nicht vernachlässigt werden, da die Entscheidungen und Ergebnisse dieser Phase eine große Tragweite über den gesamten Entwicklungsprozess besitzen und von großer Bedeutung für die erfolgreiche Datenbankenwicklung sind.

Um das zu entwickelnde Datenbanksystem in groben Zügen planen und die notwendigen Anforderungen erfüllen zu können, müssen alle relevanten Informationen erfasst werden, die für die Entwicklung von Bedeutung sind. Dazu zählen:

- Analyse der gegebenen Unternehmensstrukturen und bestehender Arbeitsprozesse, die in das Datenbanksystem eingebettet werden sollen.
- Analyse der benötigten und relevanten Daten entsprechend dem Informationsbedarf der Benutzer und Anwendungen.
- Analyse aller Anforderungen der Datenbankanwendungen wie beispielsweise Datenformate und Schnittstellen.

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:42

- Analyse der Sicherheits- und Schutzanforderungen sowie der IT-Compliance, also der Berücksichtigung relevanter Gesetze, Regeln und Normen.

Sind alle relevanten Bereiche analysiert und die wichtigsten Informationen erfasst, werden die Ergebnisse schriftlich, für alle Beteiligten verständlich in einem **Pflichtenheft** festgehalten. Diese Beschreibung enthält insbesondere auch die Datenanforderungen, also die Beschreibung aller relevanten Daten und deren Bedeutung.

Konzeptioneller Entwurf

In der zweiten Phase wird auf Basis des Pflichtenheftes und anhand der gesammelten Anforderungen ein Datenmodell entwickelt, welches sich ganz auf die fachlich konzeptionelle Seite fokussiert und den gegebenen Sachverhalt der Datenwelt noch völlig frei vom später eingesetzten Datenbanksystem und dem damit verbundenen Datenbankmodell beschreibt.

Durch die Identifikation der Instanzen, ihrer Eigenschaften und Beziehungen wird eine erste formale Beschreibung der abzubildenden Daten aus der realen Welt erstellt, die von der konkreten Realisierung in einem bestimmten System unabhängig ist. Die später zu entwerfenden Datenstrukturen in Form eines logischen Datenmodelles und das später eingesetzte Datenbankmanagementsystem spielen in dieser Phase noch keine Rolle.

Zur formellen, abstrakten Beschreibung des abzubildenden Realitätsausschnitts werden **semantische Datenmodelle** eingesetzt, die aufgrund ihrer hohen Ausdrucksstärke den Realitätsausschnitt an Daten möglichst exakt und vollständig wiedergeben können. Semantische Datenmodelle beschreiben abstrakt einen Ausschnitt der realen Welt als Menge von Objekten, zwischen denen definierte Beziehungen existieren und die durch Eigenschaften charakterisiert werden. Ein häufig verwendetes semantisches Datenmodell ist das **Entity-Relationship-Modell** (ER-Modell) nach Chen (▶ Kap. 4.3).

Die Vorgehensweise zur Erarbeitung eines konzeptionellen Datenmodells kann **Top-down** oder **Bottom-up** erfolgen. Nach dem **Prinzip der schrittweisen Verfeinerung** wird zunächst ein grobes und stark abstrahiertes Datenmodell skizziert, welches sukzessive in weitere und konkretere Teilausschnitte verfeinert wird. Hier spricht man von einem Top-down-Vorgehen. Bei einem Bottom-up-Ansatz versucht man, die Daten aus konkreten Anwendungen heraus zu beschreiben, die man in einem systematischen Integrationsprozess anschließend zu einem Gesamtmodell zusammenfügt. Das Ergebnis dieser Entwurfsphase ist ein **konzeptionelles Datenmodell** bzw. ein konzeptionelles Schema.

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:42

Logischer Entwurf

In der Phase des logischen Entwurfs wird das zuvor erstellte konzeptionelle Datenmodell in ein ausgewähltes, durch das später eingesetzte Datenbankmanagementsystem unterstütztes **logisches Datenbankmodell** überführt. Bei Einsatz eines relationalen Datenbanksystems wird beispielsweise das zuvor erstelle *Entity-Relationship-Modell* in ein **relationales Datenmodell**, also in ein **Relationenschema**, transformiert. Die Abbildung des konzeptionellen Datenmodells in ein logisches Datenmodell wird als Transformation bezeichnet. Mittels **Transformationsregeln** kann beispielsweise ein Entity-Relationship-Modell unter Einhaltung erwünschter Eigenschaften automatisiert in ein Relationenschema übertragen werden.

Das konzeptionelle wie auch das logische Datenmodell müssen erwünschte Eigenschaften erfüllen. Die Qualität der modellierten Daten hat eine große Auswirkung hinsichtlich der Anwendungsfähigkeit des späteren Datenbanksystems im betrieblichen Umfeld. Die erwünschten Eigenschaften werden bei relationalen Datenmodellen anhand von **Normalisierungsregeln** hinsichtlich ihrer Erfüllung kontrolliert und bei Bedarf in das Datenmodell eingearbeitet. Ziel ist dabei die Optimierung des Datenmodelles, der Erhöhung der Datenintegrität und die Minimierung von Redundanzen zum Erhalt der Konsistenz von Daten bei Änderungsoperationen.

Physischer Entwurf

Im physischen Datenbankentwurf wird das Ziel verfolgt, möglichst **speichereffiziente Datenstrukturen** im ausgewählten Datenbanksystem zu erreichen. Dazu werden die verfügbaren internen Speicherstrukturen des Datenbanksystems so gewählt, dass eine möglichst optimale Effizienz in Bezug auf den Ressourcenverbrauch und die Performance in der Verarbeitung und im Zugriff erreicht wird.

Für die Zielerreichung sind die seitens des ausgewählten Datenbanksystems bereitgestellten Funktionalitäten im Hinblick auf systemtechnische Gestaltungsmöglichkeiten von fundamentaler Bedeutung. Hier kommen insbesondere auch Mechanismen zum Tragen, die die Anforderungen an Datensicherheit und an Datenschutz umsetzen lassen.

In der Phase des physischen Entwurfs wird das zuvor erstellte logische Datenmodell in das **interne Schema** mit Hilfe der Datenbanksprache des ausgewählten Datenbankmanagementsystems abgebildet. Um eine effiziente Speicherung zu erreichen, muss eine geeignete Struktur der physischen Datenspeicherung festgelegt werden.

Ausschlaggebende Faktoren für die Form der physischen Datenspeicherung sind

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:42

- Zugriffspfade,
- Speicherplatzbedarf entsprechend dem analysierten Mengengerüst an zu verwaltenden Daten,
- Häufigkeit des Zugriffs und
- Änderungsfreundlichkeit.

Moderne Datenbankmanagementsysteme sind heute immer häufiger in der Lage, die Optimierung der Speicherstrukturen selbstständig vorzunehmen.

Implementierung und Wartung

In der letzten Phase des Entwicklungsprozesses wird das Datenbanksystem in Betrieb genommen und die Datenbank mit den konkreten Daten befüllt. Die Daten in der Datenbank werden während der gesamten Laufzeit über das Datenbankmanagementsystem in kontrollierter Weise abgefragt, eingefügt, geändert oder gelöscht.

Das Datenbanksystem unterliegt ab der Inbetriebnahme einer dauerhaften Wartung, um Optimierungen, Verbesserungen und Erweiterungen vorzunehmen und Fehler zu beheben. Ebenfalls müssen Anpassungen an Änderungswünsche der Benutzer und geänderte Rahmenbedingungen in das System eingepflegt werden. Kleine Änderungen können oft während des laufenden Betriebs ausgeführt werden.

Für größere Änderungen kann es notwendig sein, die Konzeption neu zu überdenken und die Ergebnisse der vorherigen Phasen nochmals auf ihre Korrektheit und Vollständigkeit zu prüfen oder sogar eine neue Konzeption und Entwicklung durchzuführen.

Die wichtigsten Aufgaben während des Datenbanksystembetriebs sind:

- **Datensicherung**: Um zu gewährleisten, dass die Daten in einem Datenbanksystem stets zur Nutzung vorliegen und im Fehlerfall schnell einen möglichst aktuellen Datenbestand wiederherstellen zu können (engl.: *Recovery*), müssen regelmäßige Datensicherungen (engl.: *Backup*) angefertigt werden.
- **Datenarchivierung**: Um den Datenbestand überschaubar und effizient zu halten, müssen nicht mehr benötigte Daten archiviert und aus der Datenbank entfernt werden. Die Aufbewahrungsfrist für archivierte Daten ist oftmals an gesetzliche Vorgaben gebunden. Beispielsweise müssen Buchhaltungs- und Finanzdaten mindestens zehn Jahre verfügbar bleiben.
- **Systemüberwachung**: Um ein ordnungsgemäßes Funktionieren des Datenbanksystems zu gewährleisten, müssen die Betriebsmittel, auf

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:42

denen das Datenbanksystem aufsetzt, überwacht werden. Faktoren wie die Auslastung der Übertragungskapazität und der Systemressourcen, beispielsweise freier Speicherplatz und CPU-Auslastung, sollten ständig überwacht und kontrolliert werden.

- **Aktualisierung** und **Optimierung**: Hardware und Datenbanksystem müssen an neue und geänderte Anforderungen angepasst werden. Das Betriebssystem und das Datenbankmanagementsystem müssen regelmäßig aktualisiert werden, um Sicherheitslücken und Fehlverhalten zu eliminieren.
- **Benutzerverwaltung** und **Zugriffskontrolle**: Die Pflege der Benutzerkonten und Berechtigungen muss regelmäßig durchgeführt und hinsichtlich der Gültigkeit geprüft werden.